

Reviewer Pack — Hypergeometric Function Moments and AC0 PARITY Circuit Compl...

Entry 392e88f2ac49 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 15:47:57 UTC

Hypergeometric Function Moments and AC0 PARITY Circuit Complexity

Entry ID: 392e88f2ac49 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 15:47:57 UTC

1. Conjecture

Field A (mathematical branch): Hypergeometric Functions **Field B** (complexity object): AC0 Parity Circuit Complexity

Statement:

[‘For every AC0 circuit C with n inputs computing the parity function, there exists a hypergeometric function f such that the sum of the moments of f up to order k is proportional to $\log^k(\text{size}(C))$ for all $k \leq \log^2(n)$.’, ‘The proportionality constant is a function of the depth d of C and grows as $O(d)$.’, ‘If C does not compute parity, then there exists no such hypergeometric function with the above property.’]

Rationale (proposer’s reasoning):

[‘Hypergeometric functions are known to be closely related to combinatorial problems involving counting. Their moments have been studied in various contexts but rarely in the context of circuit complexity.’, ‘AC0 parity is a fundamental problem in computational complexity, and finding an invariant based on hypergeometric functions could provide new insights into the structure of AC0 circuits.’, ‘If such an invariant exists, it might allow for tighter bounds on the size and depth of AC0 circuits computing parity.’]

Taxonomy category: AC0_PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 30ecae38f23528bc

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every AC0 circuit, if the ratio of the sum of moments to $\log^k(\text{size}(C))$ is within a constant factor of $O(d)$ for all $k \leq \log^2(n)$, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hypergeometric Functions" AND "AC0 Parity Circuit Complexity" - "moment of Hypergeometric function" AND AC0 - "proportional to $\log^k(\text{size}(C))$ " AND Hypergeometric Functions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def hypergeometric_moment(n, k):
        if n <= 0 or k < 0:
            return 0
        return (math.comb(n, k) / math.factorial(k)) * (1 - (k / n))

    def ac0_circuit_size(depth):
        # Simplified model for AC0 circuit size based on depth
        return 2 ** depth

    def ac0_circuit_depth(size):
        # Simplified model for AC0 circuit depth based on size
        return math.log2(size)

    n_min = 5
    n_max = 40
    k_max = 10
    trials_per_n = 30

    total_moments = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""
```

```

for _ in range(30):
    n = random.randint(n_min, n_max)
    d = ac0_circuit_depth(ac0_circuit_size(d))
    k = int(math.log2(n))

    if k > k_max:
        continue

    moments_sum = sum(hypergeometric_moment(n, i) for i in range(k + 1))
    size = ac0_circuit_size(d)
    ratio = moments_sum / (math.log(size) ** k)

    total_moments += moments_sum
    instances_tested += 1

    if not conjecture_holds and counterexample == "":
        continue

    if ratio < 0.5 * d or ratio > 2 * d:
        conjecture_holds = False
        counterexample = f"Circuit with n={n}, d={d} failed"

mean_moments = total_moments / instances_tested if instances_tested > 0 else 0

return {
    "metric_name": "Mean Moments Sum",
    "metric_value": mean_moments,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys

    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2**i - 1 for i in range(10)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        support_fraction = len([result for result in results if result["conjecture_holds"]]) / len(results)
        print(f"RESULT: SUPPORTED mean={sum(result['metric_value'] for result in results) / len(results)}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_796d4908.py", line 83, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_796d4908.py", line 46, in run_trial
    d = ac0_circuit_depth(ac0_circuit_size(d))
                                     ^
UnboundLocalError: cannot access local variable 'd' where it is not associated with a value
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Investigate the cause of the crash and attempt to run the test again to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14585
2	preregistration	ollama_remote	glm4:latest	0	0	9313
3	novelty	ollama_remote	glm4:latest	0	0	8584
4	novelty	ollama_remote	glm4:latest	0	0	8965
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12625
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8697
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11285
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8727
9	judge	ollama_remote	glm4:latest	0	0	11452

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 94234 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/392e88f2ac49.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/392e88f2ac49.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/392e88f2ac49.tar.gz` (if generated)